JDBC

By:
Mr.Satyendra Singh
Department of Computer Science & Engineering,
ABESIT College of Engineering, Ghaziabad.

JDBC Driver

. JDBC stands for Java Database Connectivity.

. JDBC is a Java API to connect and execute the query with the database.

. It is a part of JavaSE (Java Standard Edition).

. JDBC API uses JDBC drivers to connect with the database.

Java Database Connectivity

These steps are as follows:

- **Register the Driver class**:

  Class.forName("oracle.jdbc.driver.OracleDriver"); **for Oracle:**

  com.mysql.jdbc.Driver **for SQL: Create**

- **connection**:

  Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "system","password"); **for Oracle**

  dbc:mysql://localhost:3306/sonoo **for MySql**

- **Create statement:**

  Statement stmt=con.createStatement();

- **Execute queries:**

  ResultSet rs=stmt.executeQuery("select * from emp"); while(rs.next()){

  System.out.println(rs.getInt(1)+" "+rs.getString(2)); }

- **Close connection:** con.close();

Java Database Connectivity with MySQL

```
import java.sql.*;
class MysqlCon{
public static void main(String args[]){
try{
Class.forName("com.mysql.jdbc.Driver");
Connection con=DriverManager.getConnection(
"jdbc:mysql://localhost:3306/sonoo","root","root");
//here sonoo is database name, root is username and password
Statement stmt=con.createStatement();
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
con.close();
}catch(Exception e){ System.out.println(e);}
}
}
```

Connect Java Application with access with DSN

```
import java.sql.*;
class Test{
public static void main(String ar[]){
 try{
   String url="jdbc:odbc:mydsn";
   Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
   Connection c=DriverManager.getConnection(url);
   Statement st=c.createStatement();
   ResultSet rs=st.executeQuery("select * from login");
     while(rs.next()){
    System.out.println(rs.getString(1));     }
catch(Exception ee)    {  System.out.println(ee);    }
  }  }
```

Types of JDBC Driver

Type0: JDBC-ODBC Bridge Driver,
Type1:Native Driver,
Type2:Network Protocol Driver
Type3:Thin Driver

JDBC-ODBC bridge driver

- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

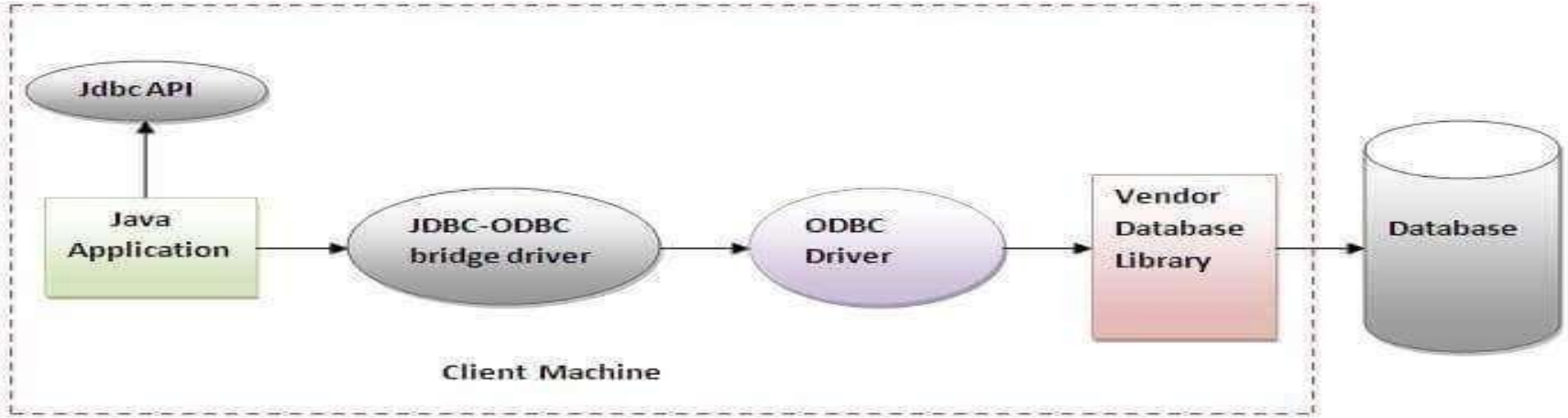

Figure- JDBC-ODBC Bridge Driver

Note: In Java 8, the JDBC-ODBC Bridge has been removed.

JDBC-ODBC bridge driver

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.
- Type-0 driver isn’t written in java, that’s why it isn’t a portable driver.

Native-API driver

- The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.
- This driver uses middlewareapplication server

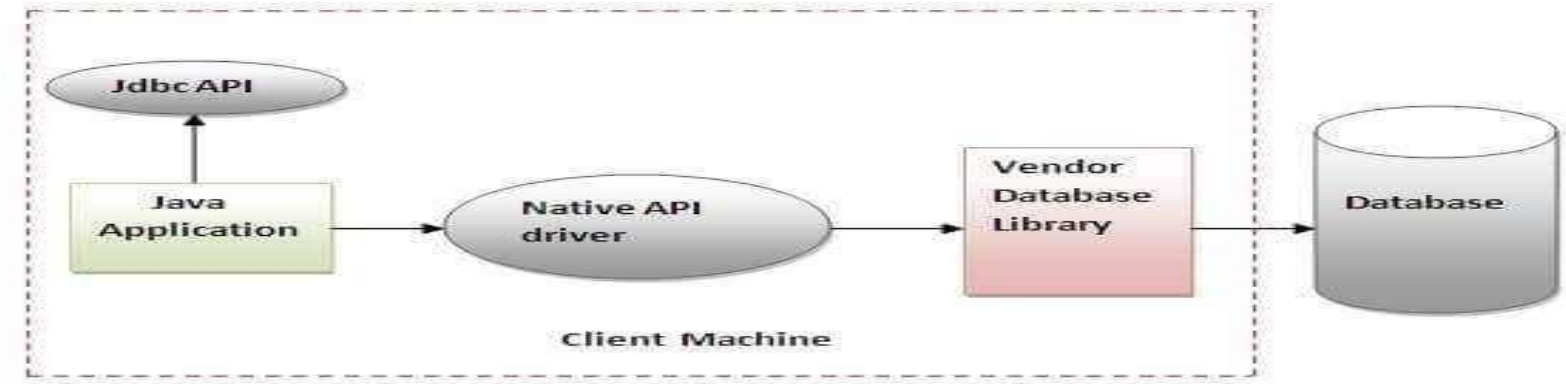

Figure- Native API Driver

Native-API driver

Advantage:

- Performance upgraded than JDBC-ODBC bridge driver.
- Type-1 drivers are fully written in Java, hence they are portable drivers.
- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantage:

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.
- Type-1 driver isn’t written in java, that’s why it isn’t a portable driver

Network Protocol driver

- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

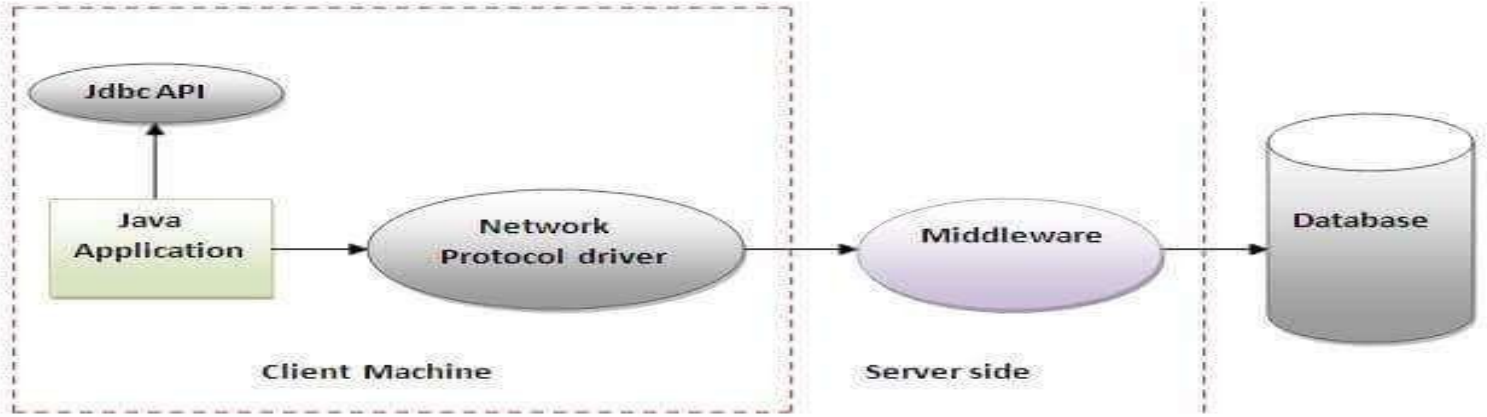

Figure- Network Protocol Driver

Advantage:

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- Java application is accessing multiple types of databases at the same time, type 2 is the preferred driver.

Disadvantages:

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

Thin driver

- The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language. Database such as **Oracle**, Sybase, or IBM

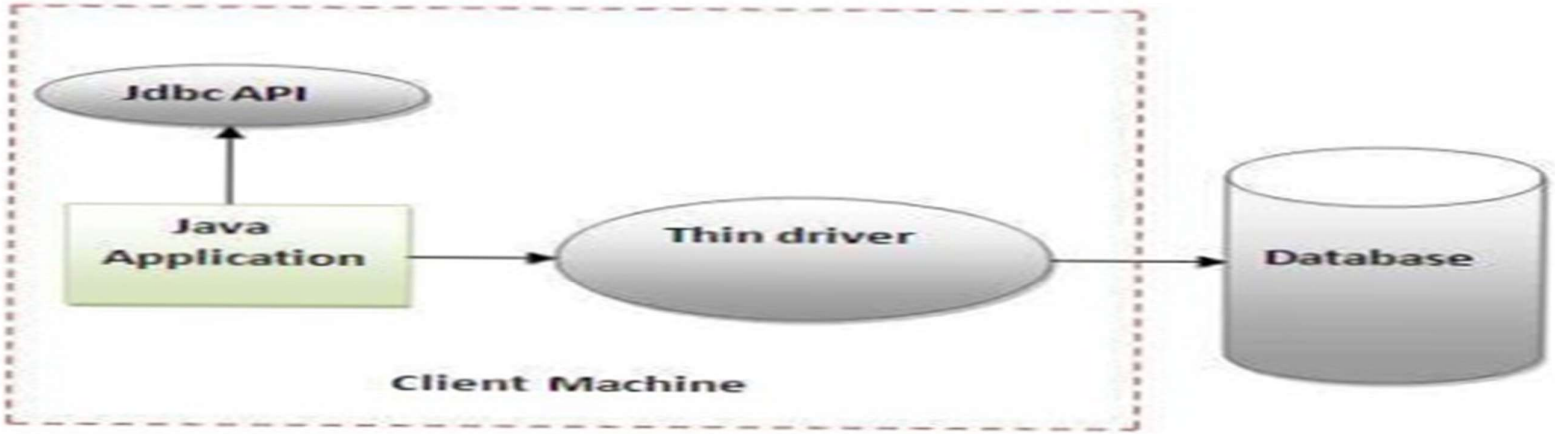

Figure- Thin Driver

Advantage:

- Better performance than all other drivers.
- No software is required at client side or server side.
- Does not require any native library and Middleware server, so no client-side or server-side installation.
- It is fully written in Java language, hence they are portable drivers.

Disadvantage:

- Drivers depend on the Database.

```
import java.sql.*;

class OracleCon {

public static void main(String args[])   {

try{          //step1 load the driver class

Class.forName("oracle.jdbc.driver.OracleDriver"); //step2 create the
connection object

Connection
con=DriverManager.getConnection(  "jdbc:oracle:thin:@localhost:1521:xe"," 
system","oracle");   //step3 create the statement object

Statement stmt=con.createStatement();    //step4 execute query

ResultSet rs=stmt.executeQuery("select * from emp");

while(rs.next())

System.out.println(rs.getInt(1)+" "+rs.getString(2)+"  "+rs.getString(3));

  //step5 close the connection object

con.close();    }    catch(Exception e)     { System.out.println(e);}

  }    }
```

Questions ?